



Department of Statistics and Data Science
MDS482-4 PROJECT

SOFTWARE REQUIREMENTS SPECIFICATION

PAWMAP

A Community-Driven Stray Animal Rescue and Welfare Platform

Prepared by

Anavadhya Devi Reg. No: 2548007
Sreya Anil Reg. No: 2548052

June 2026

Table of Contents

1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	1
1.5 References	1
2. Overall Description.....	2
2.1 Product Perspective.....	2
2.2 Product Functions.....	2
2.3 User Classes and Characteristics	3
2.4 Operating Environment.....	3
2.5 Design and Implementation Constraints	3
2.6 User Documentation.....	4
2.7 Assumptions and Dependencies	4
3. External Interface Requirements	4
3.1 User Interfaces.....	4
3.2 Hardware Interfaces.....	4
3.3 Software Interfaces.....	4
3.4 Communications Interfaces.....	5
4. System Features	5
4.1 Stray Animal Reporting Module	5
4.1.1 Description and Priority.....	5
4.1.2 Stimulus / Response Sequences	5
4.1.3 Functional Requirements.....	6
4.2 Volunteer Dispatch and Case Management Module.....	6
4.2.1 Description and Priority.....	6
4.2.2 Stimulus / Response Sequences	6
4.2.3 Functional Requirements.....	6
4.3 Medical Records Module.....	7
4.3.1 Description and Priority.....	7
4.3.2 Stimulus / Response Sequences	7
4.3.3 Functional Requirements.....	7
4.4 Adoption and Fostering Module	7
4.4.1 Description and Priority.....	7
4.4.2 Stimulus / Response Sequences	7
4.4.3 Functional Requirements.....	8
4.5 Analytics and Geospatial Intelligence Module	8
4.5.1 Description and Priority.....	8
4.5.2 Stimulus / Response Sequences	8
4.5.3 Functional Requirements.....	8
5. Non-Functional Requirements	9
5.1 Performance Requirements	9
5.2 Safety Requirements.....	9
5.3 Security Requirements	9
5.4 Software Quality Attributes.....	9
5.5 Business Rules	10
6. Requirements Specification.....	10
6.1 Objectives.....	10
6.2 Roles and Responsibilities	10
6.2.1 Project Team.....	10
6.2.2 System Roles	11

6.3 Project Timeline and Planning.....	11
7. Data Set Description and Pilot Data	12
7.1 Feasibility Study	12
7.2 Tools Requirement.....	12
7.3 Pilot Data Collection	13
7.4 Data Field Definitions	13
8. Literature Review	14
8.1 Overview	14
9. System Models	15
9.1 System Architecture Overview	15
9.2 Module Descriptions.....	15
9.2.1 Reporting Module	15
9.2.2 Volunteer Dispatch Module	15
9.2.3 Medical Records Module	15
9.2.4 Adoption Module.....	16
9.2.5 Analytics Module.....	16
Appendix A: Glossary.....	16
Appendix B: To Be Determined List.....	17

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete functional and non-functional requirements for PawMap, a community-driven mobile and web application for stray animal rescue and welfare. The document covers version 1.0 of the platform, encompassing the core reporting, volunteer coordination, medical records, adoption, and analytics subsystems. It serves as the authoritative reference for all design, development, and testing activities carried out during the MDS482-4 Project course at Christ University, Bengaluru.

1.2 Document Conventions

This document follows the IEEE Std 830-1998 SRS template. Headings are set in Times New Roman 14pt Bold; body text is Times New Roman 12pt. Requirements are labelled REQ-XX (functional) or NFR-XX (non-functional). Priority levels are assigned as High (H), Medium (M), or Low (L). All geographic measurements are in kilometres unless stated otherwise. TBD marks items not yet finalised.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

- Project supervisors and evaluators: Sections 1 through 6 in full.
- Developers (Anavadhya Devi and Sreya Anil): All sections, with particular attention to Sections 3, 4, and 5.
- NGO stakeholders: Sections 2 and 4 (Product Functions and System Features).
- Testers: Sections 4 and 5 (System Features and Non-functional Requirements).

First-time readers are advised to begin with Section 2 (Overall Description) for context before proceeding to detailed requirements in Sections 3 and 4.

1.4 Product Scope

PawMap is a cross-platform application accessible on Android, iOS, and web browsers. It enables any community member to report stray or injured animals with geo-tagged photographs, connects volunteers for rapid response, maintains caregiver-independent medical records, facilitates structured adoption, and generates geospatial intelligence for NGO decision-making. The platform is designed for initial deployment in Bengaluru, India, with architecture that supports national scale-out.

PawMap is developed under the MDS482-4 Project course, Department of Statistics and Data Science, Christ University. It applies data science, full-stack engineering, and geospatial technology to a socially significant domain.

1.5 References

1. M. Ester, H. P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," KDD-96, Portland, OR, USA, 1996, pp. 226-231.
2. S. J. Bhalla, R. Kemmers, and A. Vasques, "Stray appetites: A socio-ecological analysis of free-ranging dogs living alongside human communities in Bangalore, India," Urban Ecosystems, vol. 24, no. 6, pp. 1245-1258, 2021.
3. L. Kochumon et al., "The Paw Project: A Mobile Application for Identifying Stray Dogs," IJERT, vol. 12, no. 5, 2023.
4. M. Rajwani et al., "Enhancing Animal Rescue Efficiency Using AI for Real-Time Reporting and Assistance," Atlantis Press, 2025.
5. K. Subramaniam and H. K. Mammi, "Stray Rescue Management System (SRMS)," Int. Journal of Innovative Computing, vol. 15, no. 1, pp. 139-144, 2025.
6. H. S. Efe, G. Yildiz, and O. Yakut, "Be My Home in Smart Cities: A Mobile Application for Stray Animal Tracking, Adoption and Shelter Management," European Journal of Natural and Social Sciences, vol. 4, no. 1, 2025.
7. R. Rajkumar et al., "Paw Safe: An Extensive Technology-Driven Framework for Stray Dog Rescue, Healthcare Management, Community Engagement, and Smart Urban Governance," 2026.
8. World Health Organization and WSPA, Guidelines for Dog Population Management, WHO Press, Geneva, 2010.
9. Karuna Nidhan, Stray Animal Rescue App, 2025. [Online]. Available: <https://karunanidhan.app/>

2. Overall Description

2.1 Product Perspective

PawMap is a new, self-contained platform with no direct predecessor. It is positioned as a successor to informal coordination mechanisms -- WhatsApp groups, social media posts, and NGO helplines -- that currently dominate urban stray animal rescue in India. The platform integrates a mobile-first reporting interface, a real-time volunteer dispatch system, a shared veterinary records database, and an analytics dashboard into a unified architecture.

PawMap is composed of three subsystems: (1) a mobile application (React Native, Android/iOS) serving reporters, volunteers, and adopters; (2) a web application (React + Bootstrap 5) serving NGO administrators and veterinary professionals; and (3) a backend API layer (Django REST Framework) connected to a PostgreSQL/PostGIS spatial database and cloud task queue (Celery + Redis). External services include Google Maps API for geospatial operations and Firebase Cloud Messaging for push notifications.

2.2 Product Functions

The major functions of PawMap are summarised below:

- Geo-tagged stray animal reporting with photograph upload and severity classification.
- Atomic case-claiming and volunteer dispatch with automatic 30-minute reassignment.

- Caregiver-independent medical record management with veterinary access controls.
- Standardised five-point temperament rating aggregated into a weighted safety indicator.
- Foster-to-adopt pipeline with adopter interest queues and verified profiles.
- Geospatial hotspot detection (DBSCAN), coverage gap analysis, and volunteer reliability scoring.
- Automated monthly impact reports for NGO stakeholders.

2.3 User Classes and Characteristics

PawMap serves four distinct user classes:

User Class	Description	Technical Level	Primary Interface
Community Reporter	Any citizen who spots a stray or injured animal. May be one-time or occasional user.	Low	Mobile app
Volunteer	Registered responder who accepts and resolves rescue cases. Regular, location-aware user.	Medium	Mobile app
Veterinary Professional	Licensed vet accessing shared medical histories. Infrequent, high-trust user.	Medium	Web dashboard
NGO Administrator	Manages volunteers, oversees cases, generates reports. Frequent power user.	Medium-High	Web dashboard

2.4 Operating Environment

The PawMap platform is designed to operate in the following environment:

- Mobile: Android 10+ and iOS 14+, React Native 0.73+, minimum 2 GB RAM, GPS-enabled device, active internet connection.
- Web: Chrome 110+, Firefox 110+, Safari 16+, Edge 110+.
- Backend: Django 4.2+, Python 3.11+, PostgreSQL 15+ with PostGIS 3.3+, Redis 7+.
- Cloud: AWS EC2 (backend), AWS RDS (database), or equivalent Google Cloud Platform services.
- External APIs: Google Maps Platform (Maps JavaScript API, Geocoding API, Places API), Firebase Cloud Messaging.

2.5 Design and Implementation Constraints

- The mobile application must function with degraded GPS accuracy in dense urban areas and must cache submitted reports locally if connectivity is lost, syncing when restored.

- All personally identifiable information (PII) must be encrypted at rest (AES-256) and in transit (TLS 1.3).
- All personal data handling must comply with applicable Indian data protection standards.
- The technology stack is constrained to: Django, React, PostgreSQL/PostGIS, Celery/Redis, Google Maps API, Firebase, and scikit-learn.
- The project timeline limits scope to the five core modules described in Section 4; IoT/NFC tag integration is deferred to a future phase.

2.6 User Documentation

- In-app contextual help tooltips for all primary workflows (reporting, case claiming, record entry, adoption listing).
- NGO Administrator User Manual (PDF, delivered with the system).
- API documentation (auto-generated via DRF Spectacular) for any third-party integrations.
- Quick-start guide for volunteer onboarding (digital PDF and printable).

2.7 Assumptions and Dependencies

- Users have active smartphone devices with GPS and a data connection adequate for image upload.
- Google Maps Platform API keys will be obtained and remain within free-tier or budgeted usage limits throughout the project period.
- Firebase Cloud Messaging will be used for push notifications; delivery is subject to device-level notification permissions.
- NGO partners will supply verified volunteer lists for initial platform seeding.
- DBSCAN hotspot analysis assumes sufficient case density (minimum 50 geo-tagged reports per ward per month) for statistically significant clustering.

3. External Interface Requirements

3.1 User Interfaces

The mobile application (React Native) provides the following primary screens: Case Reporting Form (photo capture, GPS auto-fill, severity selector, species picker, description text); Active Cases Map (real-time pins, filter by status/species); Volunteer Dashboard (claimed case details, status update, checklist); Medical Record Entry (animal profile, diagnosis, treatment, vaccination log); and Adoption Listing (animal card, temperament score, foster notes, adopter interest button).

The web application (React + Bootstrap 5) provides: NGO Admin Dashboard (case volume, response KPIs, volunteer leaderboard); Analytics View (DBSCAN heatmap, coverage gap layer, trend charts); Report Generation (monthly PDF export); and Adopter Management Panel. All interfaces must be accessible on screen widths from 375 px (mobile) to 1920 px (desktop) and must conform to WCAG 2.1 AA contrast standards.

3.2 Hardware Interfaces

The mobile client interfaces with the device camera (for photo capture), GPS module (for automatic coordinate population), and push notification subsystem (via Firebase Cloud Messaging). No dedicated hardware peripherals are required. Future phases may integrate NFC readers for animal tag scanning; this interface is not in scope for version 1.0.

3.3 Software Interfaces

Component	Interface	Data Exchanged
Google Maps Platform	REST API (HTTPS)	Geocoordinates, map tiles, reverse geocoding, geofence definitions
Firebase Cloud Messaging	FCM SDK / REST API	Push notification tokens, case alert payloads
PostgreSQL + PostGIS	Internal (Django ORM + psycopg2)	All structured application data; spatial queries (ST_DWithin, ST_ClusterDBSCAN)
Celery + Redis	Internal (Celery broker)	Task queues for auto-release, SOS escalation, scheduled analytics jobs
scikit-learn (DBSCAN)	Internal (Python process)	Geo-tagged case coordinates to cluster labels and hotspot polygons

3.4 Communications Interfaces

All client-server communication uses HTTPS (TLS 1.3) over port 443. The Django REST Framework API follows RESTful conventions with JSON request/response bodies. Authentication uses JWT (JSON Web Tokens) with 60-minute access tokens and 7-day refresh tokens. Push notifications are delivered via the Firebase Cloud Messaging HTTP v1 API. WebSocket connections (Django Channels) are used for real-time case status updates on the volunteer dashboard.

4. System Features

PawMap is organised into five core functional modules. Each module is described with its priority, stimulus-response sequences, and itemised functional requirements.

4.1 Stray Animal Reporting Module

4.1.1 Description and Priority

This module allows any registered or guest community member to submit a geo-tagged rescue case within 30 seconds. Priority: HIGH.

4.1.2 Stimulus / Response Sequences

- User opens the app and taps 'Report a Stray'. The system requests camera and GPS permissions if not already granted.
- User captures or uploads a photograph. The system extracts GPS coordinates automatically or prompts manual pin-drop if GPS is unavailable.
- User selects species (dog/cat/other), severity (1-5 scale), and optionally adds a description. The system validates completeness.
- User taps 'Submit'. The system creates a case record with UUID, timestamp, coordinates, and media reference, then pushes notifications to volunteers within 5 km.

4.1.3 Functional Requirements

- REQ-01 (H): The system shall allow case submission with a single photograph, auto-populated GPS coordinates, species selection, and severity level in 30 seconds or fewer under normal network conditions.
- REQ-02 (H): The system shall store unsubmitted cases in local device cache if network connectivity is unavailable and sync them automatically when connectivity is restored.
- REQ-03 (H): Each submitted case shall be assigned a unique UUID, timestamped in UTC, and linked to a geo-tagged coordinate pair (latitude, longitude, accuracy in metres).
- REQ-04 (M): The system shall accept photographs up to 10 MB and compress them to under 500 KB before storage without visible quality loss.
- REQ-05 (M): Guest reporters (not logged in) shall be able to submit cases but shall not be able to claim or update cases.

4.2 Volunteer Dispatch and Case Management Module

4.2.1 Description and Priority

This module coordinates volunteer response to submitted cases, prevents duplication through atomic case-claiming, and auto-reassigns cases that are not actioned within 30 minutes. Priority: HIGH.

4.2.2 Stimulus / Response Sequences

- A new case is created. The system identifies active volunteers within 5 km and sends push notifications ordered by volunteer reliability score.
- Volunteer taps 'Claim Case'. The system atomically locks the case to that volunteer; all other volunteers see the case as 'In Progress'.
- 30 minutes elapse with no status update from the claiming volunteer. A Celery task auto-releases the case and re-notifies the next eligible volunteer.
- Volunteer updates case status (En Route / On Site / Resolved / Escalated). The system logs the status change with timestamp and notifies the original reporter.

4.2.3 Functional Requirements

- REQ-06 (H): The system shall implement atomic case-claiming using database-level row locking to prevent concurrent claims of the same case.
- REQ-07 (H): The system shall automatically release an unclaimed or inactive case to the next eligible volunteer after 30 minutes and log the release event.

- REQ-08 (H): Volunteers shall be able to escalate a case to SOS status, which bypasses the reliability-score queue and notifies all active NGO administrators immediately.
- REQ-09 (M): The system shall provide volunteers a real-time in-app map showing the case pin and their current location with a turn-by-turn link to Google Maps.
- REQ-10 (M): Case history shall record every status transition with volunteer ID, timestamp, and optional free-text note.

4.3 Medical Records Module

4.3.1 Description and Priority

This module maintains a continuous, caregiver-independent digital medical record for each animal registered on the platform. Priority: HIGH.

4.3.2 Stimulus / Response Sequences

- Volunteer resolves a case and creates an animal profile (species, estimated age, distinguishing features, photograph). The system generates a unique Animal ID.
- Veterinary professional logs in and searches by Animal ID. The system returns the full medical timeline.
- Vet adds a diagnosis or vaccination entry. The system appends it to the animal's record with vet credentials, date, and clinic name; the record is immediately visible to all verified users.

4.3.3 Functional Requirements

- REQ-11 (H): Each animal shall have a unique Animal ID that persists regardless of caregiver handover.
- REQ-12 (H): Medical records shall be readable by all verified NGO members and veterinary professionals; write access shall be restricted to veterinary professionals and NGO administrators.
- REQ-13 (H): The system shall maintain a complete, append-only audit trail of all medical record entries, including the author's user ID and timestamp.
- REQ-14 (M): The system shall generate vaccination reminder alerts to the current foster or NGO administrator when a due date is within 7 days.
- REQ-15 (L): The system shall support export of an individual animal's full medical history as a PDF document.

4.4 Adoption and Fostering Module

4.4.1 Description and Priority

This module facilitates structured adoption through a foster-to-adopt pipeline, adopter interest queues, and verified adopter profiles. Priority: MEDIUM.

4.4.2 Stimulus / Response Sequences

- NGO administrator marks an animal as 'Available for Adoption'. The system publishes the animal card (photo, temperament score, medical summary, foster notes) on the adoption listing.
- Registered adopter clicks 'Express Interest'. The system adds them to the adopter interest queue for that animal and notifies the NGO administrator.

- NGO administrator approves an adopter. The system creates a foster-to-adopt contract record and moves the animal to 'Adoption Pending' status.
- NGO administrator marks adoption as completed. The system archives the animal profile and increments the platform-level adoption counter.

4.4.3 Functional Requirements

- REQ-16 (M): The system shall publish adoption listings with a five-point temperament score (computed as a weighted average of all volunteer ratings), medical summary, and current foster information.
- REQ-17 (M): Adopter profiles shall require verified contact information and a completed home-readiness questionnaire before the 'Express Interest' button is activated.
- REQ-18 (M): The system shall maintain an ordered adopter interest queue per animal; the NGO administrator may reorder or remove adopters from the queue.
- REQ-19 (L): The system shall send automated follow-up surveys to confirmed adopters at 30 and 90 days post-adoption to collect welfare outcome data.

4.5 Analytics and Geospatial Intelligence Module

4.5.1 Description and Priority

This module transforms accumulated geo-tagged case data into actionable intelligence through DBSCAN hotspot detection, coverage gap analysis, and volunteer reliability scoring. Priority: MEDIUM.

4.5.2 Stimulus / Response Sequences

- The system accumulates 50 or more geo-tagged cases in a month. A scheduled Celery task runs DBSCAN clustering on case coordinates and renders hotspot polygons on the analytics map.
- NGO administrator opens the Analytics Dashboard. The system displays the hotspot heatmap, coverage gap layer (wards with cases but no responding volunteer within 3 km), and a volunteer reliability leaderboard.
- First day of each calendar month. The system auto-generates a PDF impact report (case volume, response times, adoption count, hotspot summary) and emails it to all registered NGO administrators.

4.5.3 Functional Requirements

- REQ-20 (M): The system shall apply DBSCAN spatial clustering (eps = 500 m, min_samples = 5) to geo-tagged case reports to identify statistically significant abandonment hotspots, distinguishing genuine clusters from areas of high reporter density.
- REQ-21 (M): The system shall perform a spatial join between case locations and active volunteer footprints to flag coverage gap zones, updated weekly.
- REQ-22 (M): The system shall compute a Volunteer Reliability Score (VRS) for each volunteer as a weighted composite of: response rate (40%), case completion rate (40%), and average response time (20%), updated daily.
- REQ-23 (H): The VRS shall be used to rank volunteers in the dispatch queue; only volunteers with VRS ≥ 0.6 (on a 0-1 scale) shall be eligible for SOS-tier cases.
- REQ-24 (M): The system shall generate and distribute a monthly PDF impact report on the first calendar day of each month.

5. Non-Functional Requirements

5.1 Performance Requirements

- NFR-01: Case submission (photo upload, record creation, volunteer notification dispatch) shall complete within 5 seconds on a 4G connection (minimum 10 Mbps upload).
- NFR-02: The volunteer dispatch notification shall be delivered to eligible volunteers within 10 seconds of case submission under normal server load.
- NFR-03: The backend API shall support a minimum of 500 concurrent users without degradation in response time beyond 2 seconds per request.
- NFR-04: DBSCAN clustering over 10,000 case records shall complete within 60 seconds on the scheduled analytics worker.
- NFR-05: Map tile loading on the volunteer case map shall complete within 3 seconds on a 4G connection.

5.2 Safety Requirements

- NFR-06: Animal behavioural safety data (temperament scores) shall be prominently displayed at the top of each active case card viewed by volunteers, with a colour-coded risk indicator (Green: safe, Amber: caution, Red: high risk).
- NFR-07: SOS-escalated cases shall trigger audio and vibration alerts on the NGO administrator's device in addition to push notification.
- NFR-08: A volunteer's live GPS location shall not be visible to community reporters or unverified users; location data is restricted to NGO administrators and the volunteer themselves.

5.3 Security Requirements

- NFR-09: All data in transit shall be encrypted using TLS 1.3; all data at rest shall be encrypted using AES-256.
- NFR-10: User authentication shall use JWT with 60-minute access token expiry and mandatory re-authentication after 7 days.
- NFR-11: Veterinary professional and NGO administrator roles shall require manual verification by a platform administrator before activation.
- NFR-12: Photographs of animals shall be watermarked with the platform logo before display to prevent unauthorised reuse.
- NFR-13: The system shall log all administrative actions (role changes, record deletions, report access) in a tamper-evident audit log retained for 12 months.

5.4 Software Quality Attributes

- Reliability: The backend API shall maintain 99.5% uptime measured monthly, excluding scheduled maintenance windows.
- Usability: A first-time reporter shall be able to submit a case without any tutorial or instruction in under 30 seconds, validated by pilot user testing with 10 participants.

- Scalability: The architecture shall support horizontal scaling of the Django application tier via containerisation (Docker) to accommodate user growth without re-engineering.
- Maintainability: All backend modules shall achieve minimum 80% unit test coverage; API endpoints shall be documented via OpenAPI 3.0 specification.
- Portability: The mobile application shall compile and run identically on Android 10+ and iOS 14+ without platform-specific code paths for core features.
- Accessibility: Web and mobile interfaces shall meet WCAG 2.1 Level AA contrast and font-size standards.

5.5 Business Rules

- A case may be claimed by only one volunteer at a time; the claiming system shall use database-level locking.
- Temperament scores are aggregated as a weighted average, with more recent ratings weighted 1.5x relative to ratings older than 30 days.
- Volunteers with a VRS below 0.4 for three consecutive months shall have their accounts flagged for NGO administrator review.
- Adoption listings shall not be published unless the animal has at minimum one completed vaccination entry in its medical record.
- Monthly impact reports shall be auto-generated on the first calendar day of each month; NGO administrators may also trigger manual generation at any time.

6. Requirements Specification

6.1 Objectives

The following six formal objectives drive all functional and non-functional requirements in this document:

10. To enable any community member to report a stray or injured animal within 30 seconds by submitting a geo-tagged photograph and basic case details through the mobile or web interface.
11. To eliminate duplicate and missed responses through an atomic case-claiming system that locks a case to the first volunteer who accepts it, with automatic reassignment after 30 minutes of inactivity.
12. To maintain a continuous, caregiver-independent medical record for each animal, accessible by all verified veterinary professionals and NGO members on the platform.
13. To provide a standardised five-point temperament rating system that aggregates multi-volunteer input into a weighted safety indicator, reducing risk for first responders.
14. To facilitate structured adoption through a foster-to-adopt pipeline, adopter interest queues, and verified adopter profiles that match animals to suitable permanent homes.
15. To generate geo-analytical insights including abandonment hotspot heatmaps using DBSCAN spatial clustering and automated monthly impact reports for NGO use.

6.2 Roles and Responsibilities

6.2.1 Project Team

Team Member	Role	Responsibilities
Anavadhya Devi Reg. No: 2548007	Lead Data Scientist / Backend Developer	DBSCAN hotspot detection, coverage gap analysis, volunteer reliability scoring; Django REST API; PostgreSQL/PostGIS schema design; pilot dataset generation.
Sreya Anil Reg. No: 2548052	Lead Frontend / Mobile Developer	React Native mobile app; React web dashboard; Bootstrap 5 UI; Google Maps API integration; Firebase Cloud Messaging; adoption and reporting modules.

6.2.2 System Roles

System Role	Permissions	Primary Interface
Community Reporter (Guest)	Submit cases; view public adoption listings.	Mobile app
Registered Reporter	All Guest permissions; track own submitted cases; receive case resolution notifications.	Mobile app
Volunteer	All Registered Reporter permissions; claim and update cases; enter temperament ratings; add foster notes.	Mobile app
Veterinary Professional	Read all animal medical records; create and update medical entries for any animal.	Web dashboard
NGO Administrator	All Volunteer permissions; manage volunteer accounts; publish adoption listings; generate reports; access analytics.	Web dashboard

6.3 Project Timeline and Planning

Phase	Weeks	Deliverables
Project Proposal	1-2	Project proposal document; domain research; literature survey initiated.
Requirement Analysis	3-4	SRS document (this document); pilot dataset (generated, 100 samples); literature review table.
System Design	5-6	ER diagram; system architecture diagram; API schema (OpenAPI 3.0); UI wireframes.
Development -- Core Modules	7-10	Reporting module; volunteer dispatch; medical records module; basic mobile app build.

Phase	Weeks	Deliverables
Development -- Advanced Modules	11-13	Adoption pipeline; DBSCAN analytics; coverage gap analysis; reliability scoring.
Integration and Testing	14-15	End-to-end integration; unit and API testing (80% coverage target); user acceptance testing with NGO partner.
Final Submission	16	Deployed prototype; final report; PDF and CSV submission.

Estimated Tool Costs: Google Maps Platform -- free tier up to \$200/month credit (sufficient for pilot scale); Firebase Cloud Messaging -- free tier; AWS EC2 t3.small + RDS db.t3.micro -- approximately \$30-50/month (development and staging environment only).

7. Data Set Description and Pilot Data

7.1 Feasibility Study

The proposed technology stack has been evaluated for feasibility within the project timeline. Django, React Native, PostgreSQL with PostGIS, and scikit-learn are mature, well-documented open-source frameworks with active community support. The team has prior coursework exposure to Python, SQL, and React. Google Maps Platform and Firebase offer free-tier credits adequate for pilot-scale testing (under 500 users). Cloud hosting on AWS Free Tier covers development and staging environments. The five-module scope is achievable within 16 weeks with the two-person team given the defined role split in Section 6.2.1.

7.2 Tools Requirement

Tool / Technology	Version	Purpose	Status
Django + DRF	4.2 / 3.14	Backend API and application logic	Confirmed
React Native	0.73	Cross-platform mobile application	Confirmed
React + Bootstrap 5	18.x / 5.3	Web dashboard	Confirmed
PostgreSQL + PostGIS	15 / 3.3	Spatial relational database	Confirmed
Celery + Redis	5.3 / 7.2	Background task queue	Confirmed
Google Maps Platform	Current	Geospatial mapping and geofencing	API key pending
Firebase Cloud Messaging	Current	Push notifications	Confirmed
scikit-learn	1.4	DBSCAN clustering and reliability scoring	Confirmed

Tool / Technology	Version	Purpose	Status
Docker	24.x	Containerisation and deployment	Confirmed
JabRef	5.x	Reference management for literature review	Confirmed

7.3 Pilot Data Collection

A synthetic pilot dataset has been generated to simulate rescue reports. The dataset includes the following fields: date and time of sighting; ward/neighbourhood; latitude and longitude; species (Dog/Cat/Other); estimated age (Puppy/Kitten/Adult/Senior); injury severity (1 = healthy, 5 = critical); injury type; number of animals at location; bystander action taken (none/fed/contacted NGO); reporter age group; and reporter prior rescue experience (yes/no). The dataset reflects realistic geographic and temporal distributions for Bengaluru urban wards.

A minimum of 100 records have been generated as the pilot dataset. This dataset will be used to: (1) validate the DBSCAN clustering pipeline with real geographic distribution data; (2) test the coverage gap analysis against known volunteer footprints; and (3) seed the platform for user acceptance testing.

The pilot dataset will be exported as a CSV file (pilot_data_pawmap.csv) and submitted alongside this SRS document as required by the course deliverable specification.

7.4 Data Field Definitions

Field Name	Type	Description	Used In
case_id	UUID	Unique identifier for each rescue report	All modules
timestamp_utc	Datetime	UTC timestamp of report submission	Hotspot detection, reliability scoring
latitude / longitude	Float	Geo-coordinates of the sighting	DBSCAN, coverage gap, map display
species	Enum	Dog, Cat, or Other	Case filtering, hotspot disaggregation
severity	Int (1-5)	Reporter-assigned injury severity	Volunteer dispatch priority, SOS trigger
ward_name	String	Administrative ward derived by reverse geocoding	Coverage gap analysis, reports
volunteer_id	UUID	ID of assigned volunteer (nullable)	Reliability scoring, dispatch history
response_time_min	Float	Minutes between case creation and volunteer claim	Volunteer Reliability Score

Field Name	Type	Description	Used In
resolution_status	Enum	Resolved, Escalated, Unresolved	Impact reporting, coverage gap
animal_id	UUID	Linked animal profile ID (assigned post-rescue)	Medical records, adoption module

8. Literature Review

8.1 Overview

The following table summarises recent and relevant works in the domain of stray animal welfare technology, geospatial coordination systems, and AI-assisted rescue platforms. References are managed using JabRef reference management software.

Year	Author / Company	Techniques / Algorithm	Gap or Drawback
2010	WHO & WSPA	Operational guidelines for municipal dog population management; no digital component.	Prescriptive policy document; no technology integration; no community-participation mechanism.
2021	Bhalla, Kemmers & Vasques	Socio-ecological field study of free-ranging dogs in Bengaluru; ethnographic and GIS mapping.	Descriptive research only; no real-time reporting system; no volunteer coordination.
2023	Kochumon et al.	Mobile app for stray dog identification using image recognition; GPS tagging.	No volunteer dispatch; no medical records; no adoption pipeline; limited to identification.
2025	Rajwani et al.	AI-based real-time reporting with image classification for injury severity detection.	No geospatial clustering; no volunteer management; standalone reporting without case lifecycle.
2025	Subramaniam & Mammi	Stray Rescue Management System (SRMS) -- centralised NGO case tracking.	No community reporting; no mobile-first interface; no data analytics or hotspot mapping.
2025	Efe, Yildiz & Yakut	Smart city mobile app for stray animal tracking, adoption listings, and shelter management.	No volunteer reliability scoring; no DBSCAN analytics; no foster pipeline.
2025	Karuna Nidhan	Community rescue app with NGO integration; India-focused.	Closed-source; limited spatial analytics; no temperament safety rating; no NGO reporting.

Year	Author / Company	Techniques / Algorithm	Gap or Drawback
2026	Rajkumar et al.	Paw Safe -- comprehensive framework integrating AI, IoT, and urban governance for stray dog rescue.	Theoretical framework; no deployed prototype; no spatial clustering or volunteer scoring described.

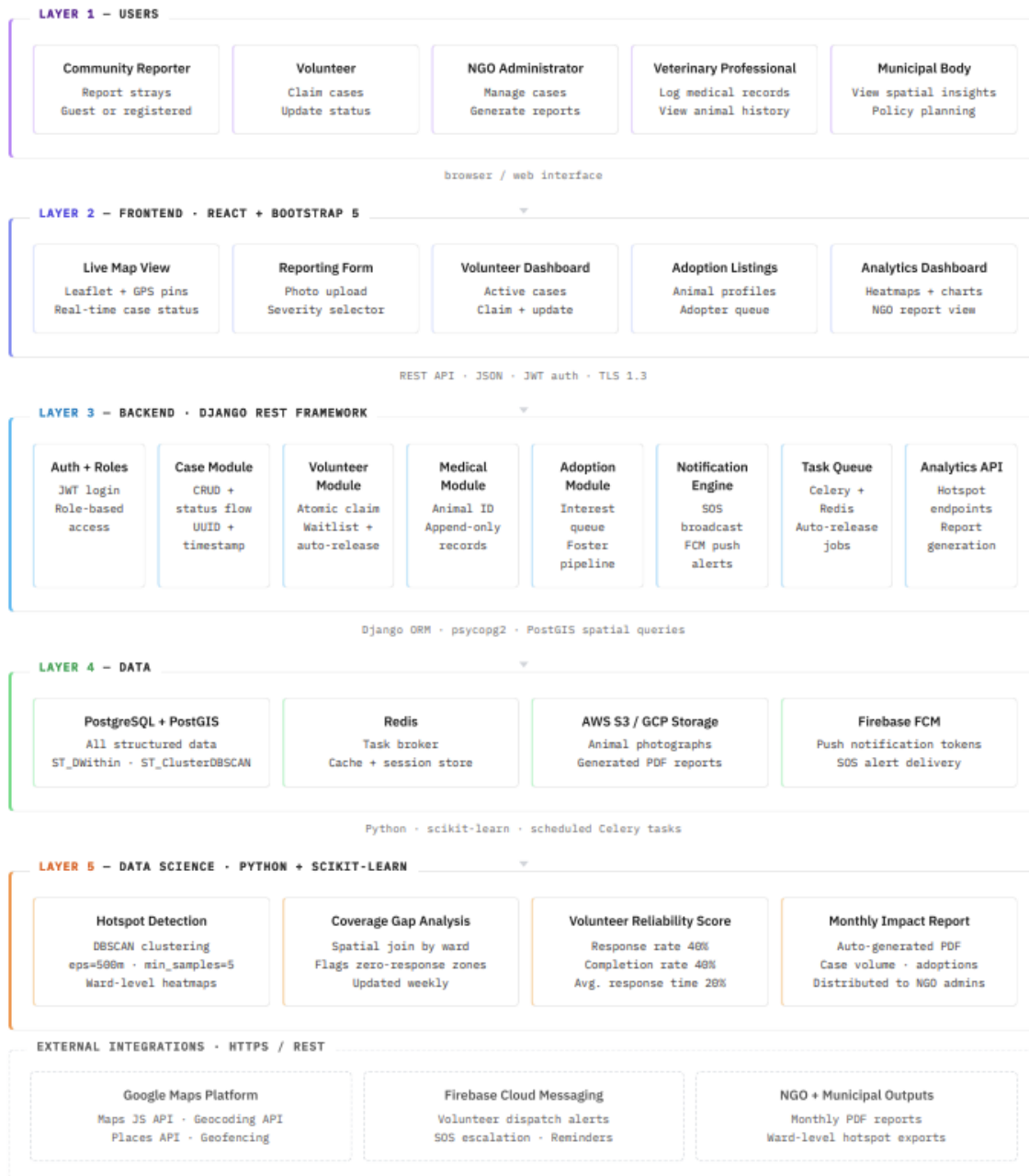
PawMap's distinguishing contributions relative to existing work are: (1) atomic volunteer dispatch with reliability scoring and automated reassignment; (2) DBSCAN-based geo-intelligence calibrated for Indian urban ward geographies; (3) a foster-to-adopt pipeline with temperament safety aggregation; and (4) a platform designed from the outset for NGO co-governance.

9. System Models

9.1 System Architecture Overview

PawMap follows a three-tier client-server architecture. The presentation tier consists of the React Native mobile application and the React web dashboard. The logic tier comprises the Django REST Framework API server, the Celery worker pool, and the scikit-learn analytics processes. The data tier comprises PostgreSQL + PostGIS for structured and spatial data, Redis for task queuing and caching, and cloud object storage (AWS S3 or GCP Cloud Storage) for image files.

External services (Google Maps Platform, Firebase Cloud Messaging) interface exclusively with the logic tier via HTTPS. The NGO administrator accesses a read-only analytics layer (web dashboard) that queries pre-computed views in the database to avoid impacting API performance.



9.2 Module Descriptions

9.2.1 Reporting Module

Input: User photograph, device GPS coordinates, species selection, severity rating (1-5), optional text description. Process: Validate and compress photograph; reverse-geocode coordinates to ward name via Google Maps Geocoding API; create Case record (UUID, timestamp, coordinates, ward, severity, media

reference); publish notification job to Celery queue. Output: Case ID returned to user; push notification dispatched to eligible volunteers within 5 km.

9.2.2 Volunteer Dispatch Module

Input: New case event from Celery queue; volunteer GPS locations; volunteer reliability scores. Process: Query PostGIS for active volunteers within 5 km of case coordinates; rank by VRS descending; dispatch FCM push notification to top-ranked volunteers; on claim event, apply row-level lock to case record and assign volunteer ID; start 30-minute auto-release Celery countdown task; on status update, cancel countdown task and log transition. Output: Case assigned to volunteer; assignment visible to NGO administrators and original reporter; auto-release triggered on inactivity.

9.2.3 Medical Records Module

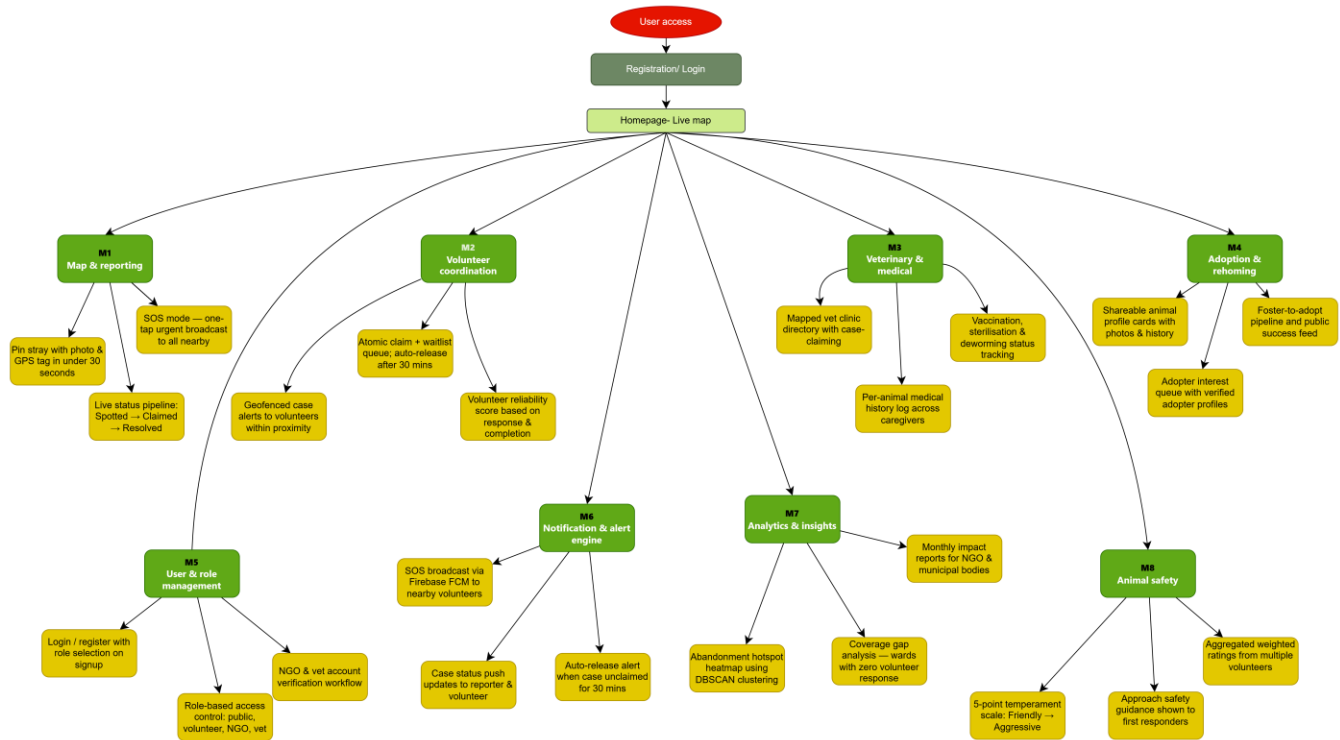
Input: Animal profile data (photograph, species, estimated age, distinguishing features) from resolving volunteer; subsequent medical entries (diagnosis, treatment, vaccination, weight) from veterinary professional. Process: Generate unique Animal ID on first profile creation; append each medical entry to the animal's timeline with author credentials and timestamp; trigger vaccination reminder jobs to Celery scheduler. Output: Persistent animal medical timeline accessible to all verified users; vaccination reminders dispatched 7 days before due date; PDF export on request.

9.2.4 Adoption Module

Input: NGO administrator action to publish animal as 'Available for Adoption'; adopter profile and home-readiness questionnaire; adopter 'Express Interest' action. Process: Validate animal has at least one vaccination record; compute and display weighted temperament score; add adopter to ordered interest queue; on administrator approval, create adoption record and transition animal status; schedule 30- and 90-day welfare follow-up surveys. Output: Public adoption card; ordered adopter queue; adoption record; automated follow-up survey emails.

9.2.5 Analytics Module

Input: Accumulated geo-tagged case records (latitude, longitude, timestamp, species, severity, ward); volunteer location histories; resolution statuses. Process: Weekly Celery task runs DBSCAN ($\text{eps} = 500$ m, $\text{min_samples} = 5$) on case coordinates; spatial join identifies coverage gap zones (cases with no volunteer response within 3 km); daily job recomputes VRS for each volunteer; first-of-month job generates PDF impact report. Output: Hotspot polygons rendered on analytics map; coverage gap ward layer; VRS leaderboard; monthly impact PDF distributed by email.



Appendix A: Glossary

Term	Definition
DBSCAN	Density-Based Spatial Clustering of Applications with Noise -- an algorithm that identifies clusters of arbitrary shape in spatial data, used in PawMap for abandonment hotspot detection.
VRS	Volunteer Reliability Score -- a weighted composite metric (response rate 40%, completion rate 40%, response time 20%) used to rank volunteers for case dispatch.
SOS Case	A rescue case escalated by a volunteer to indicate immediate danger to an animal or human, triggering priority NGO administrator notification.
Foster-to-Adopt	A transitional arrangement in which an adopter temporarily fosters an animal before formal adoption is finalised.
Geo-fencing	The use of GPS-defined virtual boundaries to trigger location-aware push notifications when a volunteer enters or exits a defined radius around a case.
PostGIS	A spatial extension to PostgreSQL that adds support for geographic objects and spatial query functions (e.g., ST_DWithin for radius queries, ST_ClusterDBSCAN for clustering).
JWT	JSON Web Token -- a compact, URL-safe means of representing authentication claims between the client and API server.

Term	Definition
FCM	Firestore Cloud Messaging -- Google's cross-platform push notification service used for volunteer dispatch alerts and SOS escalation.
Ward	An administrative subdivision of an Indian city used by local governing bodies for service delivery planning.
TBD	To Be Determined -- a placeholder used in this SRS for requirements or details not yet finalised.

Appendix B: To Be Determined List

16. TBD-01: Exact Google Maps Platform API key procurement timeline and monthly budget allocation.
17. TBD-02: Selection of cloud provider (AWS vs Google Cloud Platform) and final server tier specifications.
18. TBD-03: NGO partner organisation name and signed MOU for pilot user acceptance testing.
19. TBD-04: Specific DBSCAN hyperparameter tuning (eps and min_samples) based on pilot dataset geographic distribution.
20. TBD-05: Home-readiness questionnaire content (to be designed with NGO partner input in Week 5).